

assignment

October 10, 2025

1 Programming Assignment: Image Representations

In this problem, we will study how different representations of images can affect the performance of a nearest neighbor classifier.

We will experiment with CIFAR-10 data set, which has 50,000 training images and 10,000 test images, with ten different classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). The images are in color, of size 32×32 .

We will investigate the following representations: 1. Raw pixel space 2. Histogram-of-gradients (HOG) features 3. Convolutional Neural Network (CNN) features * Intermediate-level and high-level features extracted from a **pretrained** CNN * Intermediate-level and high-level features extracted from a **randomly initialized** CNN

Instructions

1. Install the required libraries
 - Install the CPU version of PyTorch and torchvision
 - A GPU is **not required** for this assignment.
 - For more specific instructions related to your machine, please refer to: <https://pytorch.org/get-started/locally/>.
 - Install scikit-learn **1.0.2**
 - Note that in version 1.2.2, KNN classifier with L2 distance behaves weirdly. For example, testing training samples will cause an error. Query results vary a lot over different runs. Classification accuracy is close to a random guess. We would recommend using version 1.0.2 instead.
 - Install scikit-image, numpy, os, matplotlib, tqdm, torchinfo, etc.
 - Use `pip install` to install any other required library.
2. We have provided you with a number of functions and scripts in the hopes of alleviating some tedious or error-prone sections of the implementation. You are free to modify them if necessary. The provided files include:
 - `dataset.py`
 - Functions used to download, load and visualize a dataset
 - `extract_feature.py`
 - Functions to extract a variety of features, including raw pixels, HoG features and CNN features.
 - `vgg_network.py`
 - Class defining the VGG-11 architecture, functions to load a pretrained VGG model and test it.

- `path.py`
 - Paths of directories to save the dataset, features, models, and figures. Feel free to reconfigure it.
3. Submission
- This notebook includes various things for you to do.

1.1 1. Load the data

1.1.1 Download the data set

We will be using the [CIFAR-10](#) data set for our experiments. Before getting started, run the following block to download the data set. The data set will be saved to directory `/datasets`. You can change the default download path in `path.py`.

```
[1]: from dataset import download_cifar10_dataset
dataset = download_cifar10_dataset()
```

```
Dataset Downloading ...
Downloaded CIFAR-10 dataset to /Users/twilight/Downloads/cifar-
representations/datasets
```

1.1.2 Load in the data set

Run the following block to load the training images, training labels, test images, and test labels as `x_train`, `y_train`, `x_test`, `y_test` respectively. The code will print out their shapes for you.

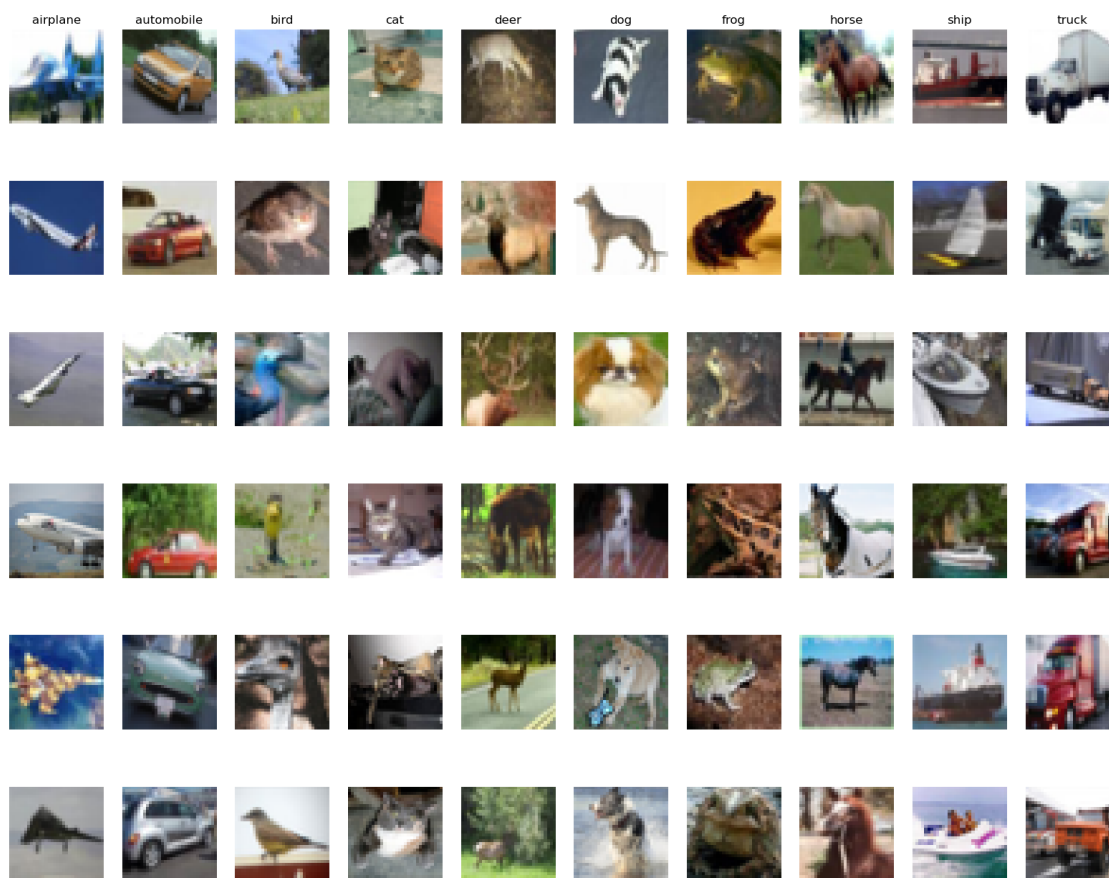
```
[2]: from dataset import load_dataset_splits
x_train, y_train, x_test, y_test = load_dataset_splits()
```

```
=====> CIFAR-10 dataset loaded
Training set data shape: (50000, 3, 32, 32)
Training set label shape: (50000,)
Test set data shape: (10000, 3, 32, 32)
Test set label shape: (10000,)
```

1.1.3 Visualize the data

Run the following block to display several randomly-sampled images from each of the ten classes.

```
[3]: from dataset import visualize_cifar_data
visualize_cifar_data(images=x_train.transpose(0, 2, 3, 1), labels=y_train,
↳ samples_per_class=6)
```



1.2 2. Nearest neighbor classification on CIFAR-10

In this part, we will investigate the classification problem by training and testing a nearest neighbor classifier on CIFAR-10 dataset with **Euclidean (L2) distance**.

Function `run_nearest_neighbor`, shown in the next cell, takes a labeled training set (`x_train`, `y_train`) and test set (`x_test`, `y_test`), and applies 1-nearest neighbor classification to the test points, using `KNeighborsClassifier` from `sklearn`. It prints out the accuracy on the test set and returns the classifier.

```
[4]: from sklearn.neighbors import KNeighborsClassifier

def run_nearest_neighbor(x_train, y_train, x_test, y_test):
    # create classifier
    nn_classifier = KNeighborsClassifier(n_neighbors=1, algorithm='auto')

    # train
    nn_classifier.fit(x_train, y_train)

    # test and report accuracy
```

```
test_acc = nn_classifier.score(x_test, y_test)
print("Nearest neighbor accuracy on the test set: %f"%test_acc)

return nn_classifier
```

1.2.1 Nearest neighbor on raw pixels

Now we do nearest neighbor classification in the raw pixel space.

We have provided you with a function `compute_or_load_features` in `extract_feature.py` to extract the features from the training and test images. You can specify the feature extraction method by setting parameter `feature_type` to one of the options in `['raw_pixel', 'hog', 'pretrained_cnn', 'random_cnn']`. When the features are extracted by a CNN, you can further specify at which layer you are extracting the features by setting parameter `layer` to one of the options in `['last_conv', 'last_fc']`. In practice, extracting features from a large data set could be time consuming. To avoid repeated computation, this function will only compute the features once and store them in directory `/features` (you can change this in `path.py`). If being called later, the function will search for the existing feature files and directly load them into memory.

Call function `compute_or_load_features` to extract the raw pixels as features and call function `run_knn` to train and test nearest neighbor classifier in this feature space. To do: Report the test accuracy you get.

```
[5]: from extract_feature import compute_or_load_features

# compute or load features
raw_pixel_train_features, raw_pixel_test_features = \
    compute_or_load_features(x_train, x_test, "raw_pixel")

# run knn
raw_pixel_knn_classifier = run_nearest_neighbor(raw_pixel_train_features, \
    y_train, raw_pixel_test_features, y_test)
```

```
=====> Loaded train and test features from /Users/twilight/Downloads/cifar-
representations/features/raw_pixel.pkl
Training feature shape: (50000, 3072)
Test feature shape: (10000, 3072)
Nearest neighbor accuracy on the test set: 0.353900
```

To get a sense of how the images are distributed locally in the pixel space, we can look at their nearest neighbors.

To do: Write code in the following block to do the following. Make your code modular so that it can be re-used for the representations we will consider. * Show the first five images in the test set whose label is correctly predicted by 1-NN, and show the nearest neighbor (in the training set) of each of these images. * Show the first five images in the test set whose label is incorrectly predicted by 1-NN, and show the nearest neighbor (in the training set) of each of the images.

```
[ ]: ### Your code here
raw_predicted = raw_pixel_knn_classifier.predict(raw_pixel_test_features)
```

```
[28]: from sklearn.neighbors import NearestNeighbors

def get_nearest_neighbors(x_train, x_test, n_neighbors=1):
    # create NearestNeighbors model
    nn_model = NearestNeighbors(n_neighbors=n_neighbors, algorithm='auto')

    # fit the model on training data
    nn_model.fit(x_train)

    # find the nearest neighbors for each test sample
    distances, indices = nn_model.kneighbors(x_test)

    return distances, indices

[29]: dis_raw, ind_raw = get_nearest_neighbors(raw_pixel_train_features,
↳ raw_pixel_test_features, n_neighbors=1)

[42]: import matplotlib.pyplot as plt
import numpy as np

def show_correct(x_train, x_test, y_pred, y_test, nn1_idx, samples_per_class=5):
    cifar_classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
↳ 'frog', 'horse', 'ship', 'truck']
    num_classes = len(cifar_classes)

    # set default size of plots
    plt.rcParams['figure.figsize'] = (20.0, 10.0)

    fig1, axes1 = plt.subplots(samples_per_class, num_classes)
    fig2, axes2 = plt.subplots(samples_per_class, num_classes)

    for cls_index, cls_name in enumerate(cifar_classes):
        corrected_predicted_idx = np.where((y_pred == cls_index) & (y_test ==
↳ cls_index))[0]
        for i, idx in enumerate(corrected_predicted_idx[:samples_per_class]):
            axes1[i, cls_index].imshow(x_test[idx] / 255.0)
            axes1[i, cls_index].axis('off')
            axes2[i, cls_index].imshow(x_train[nn1_idx[idx]].squeeze() / 255.0)
            axes2[i, cls_index].axis('off')
            if i == 0:
                axes1[i, cls_index].set_title(cls_name)
                axes2[i, cls_index].set_title(cls_name)

    fig1.suptitle('Correctly Classified Test Images')
    fig2.suptitle('Nearest Neighbor Train Images')
```

```

plt.show()

def show_incorrect(x_train, x_test, y_pred, y_test, nn1_idx,
    ↪samples_per_class=5):
    cifar_classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
    ↪'frog', 'horse', 'ship', 'truck']
    num_classes = len(cifar_classes)

    # set default size of plots
    plt.rcParams['figure.figsize'] = (20.0, 10.0)

    fig1, axes1 = plt.subplots(samples_per_class, num_classes)
    fig2, axes2 = plt.subplots(samples_per_class, num_classes)

    for cls_index, cls_name in enumerate(cifar_classes):
        incorrected_predicted_idx = np.where((y_pred != cls_index) & (y_test ==
    ↪cls_index))[0]
        for i, idx in enumerate(incorrected_predicted_idx[:samples_per_class]):
            axes1[i, cls_index].imshow(x_test[idx] / 255.0)
            axes1[i, cls_index].axis('off')
            axes2[i, cls_index].imshow(x_train[nn1_idx[idx]].squeeze() / 255.0)
            axes2[i, cls_index].axis('off')
            if i == 0:
                axes1[i, cls_index].set_title(cls_name)
                axes2[i, cls_index].set_title(cls_name)

        fig1.suptitle('Incorrectly Classified Test Images')
        fig2.suptitle('Nearest Neighbor Train Images')
    plt.show()

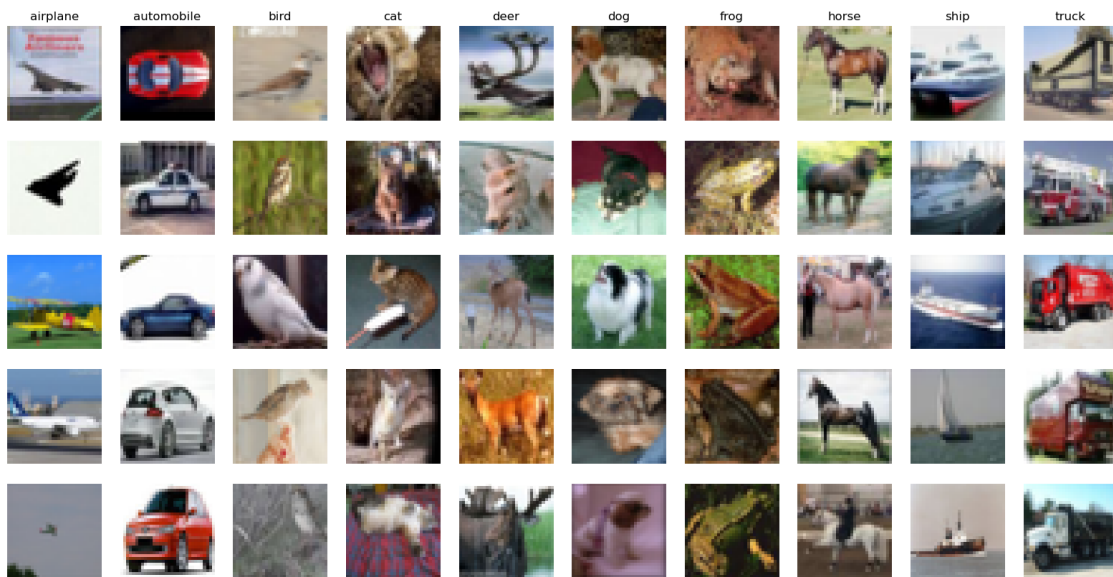
```

```

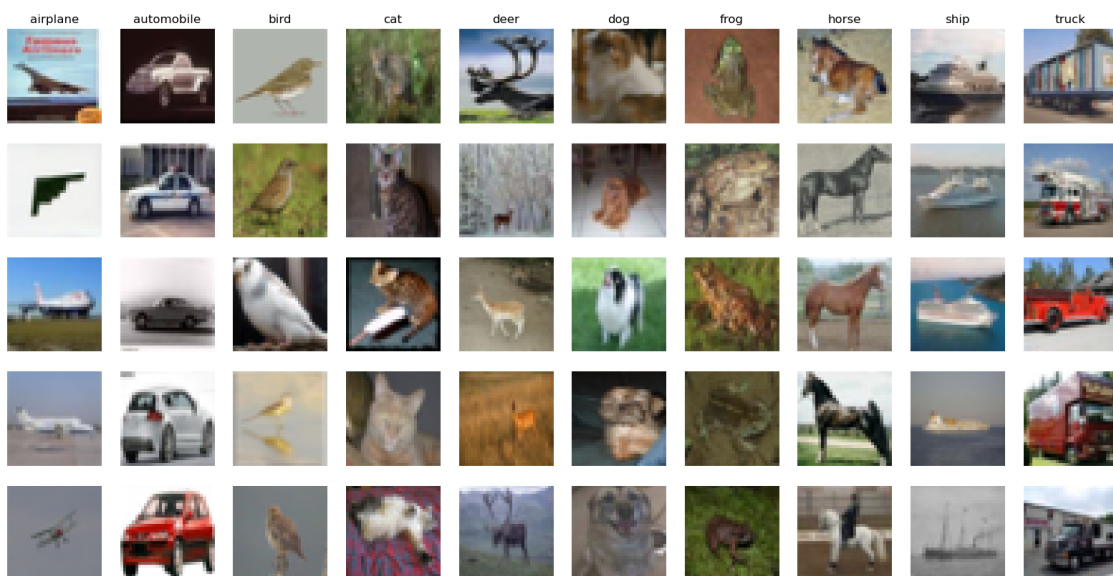
[41]: show_correct(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
    ↪raw_predicted, y_test, ind_raw, samples_per_class=5)

```

Correctly Classified Test Images

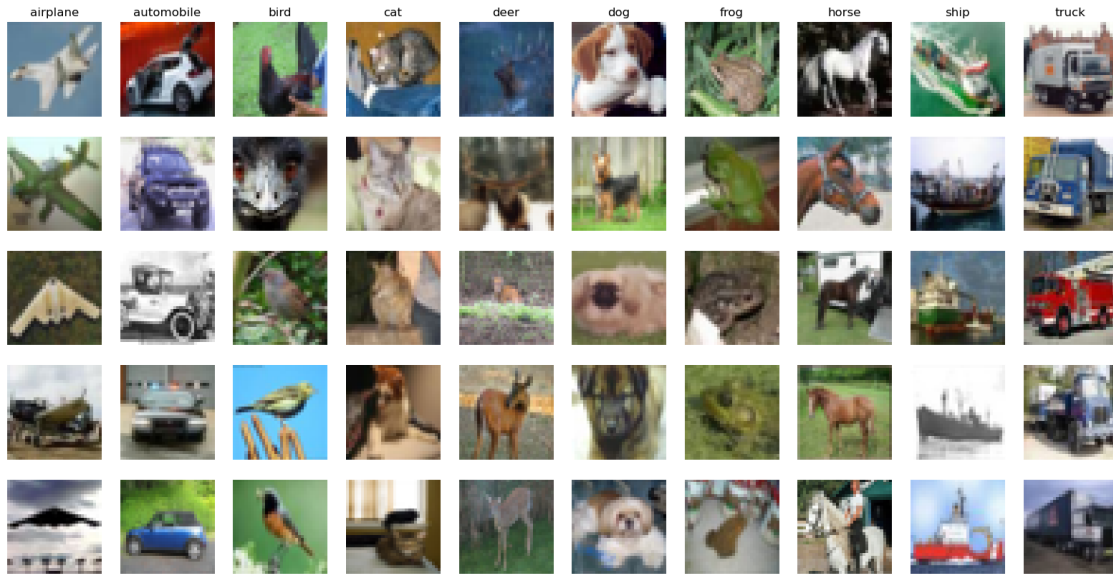


Nearest Neighbor Train Images

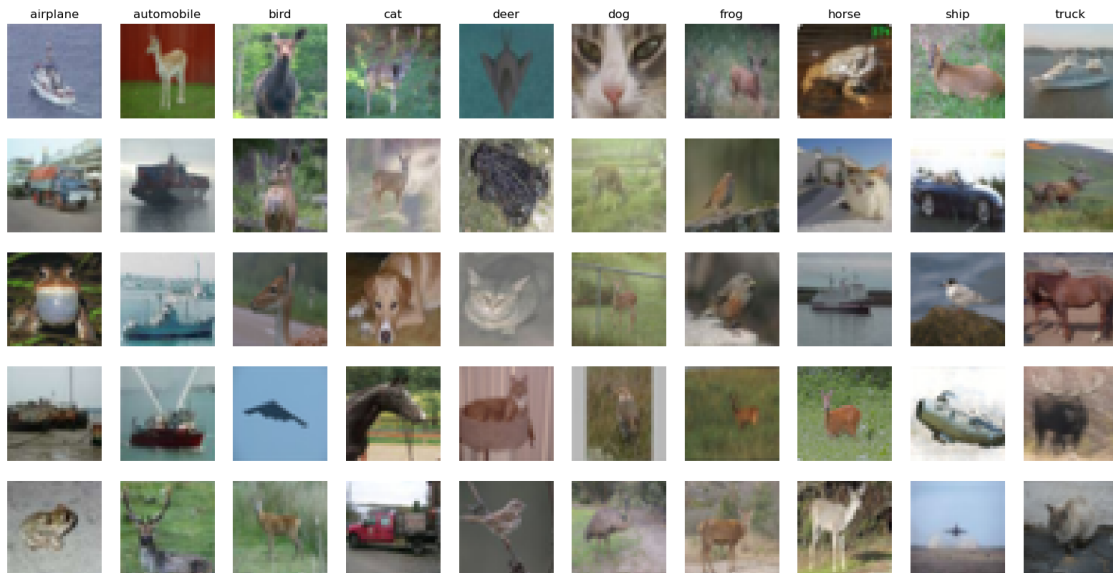


```
[43]: show_incorrect(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
    ↪ raw_predicted, y_test, ind_raw, samples_per_class=5)
```

Incorrectly Classified Test Images



Nearest Neighbor Train Images



1.2.2 Nearest neighbor on HOG features

The HOG (Histogram of Oriented Gradients) descriptor computes local statistics of gradients in an image and uses them as feature representations. Train and test a nearest neighbor classifier on HOG features. To do: Report the test accuracy. As with the raw pixel representation, show the first five correctly classified images (and their nearest neighbors) and the five first incorrectly

classified.

```
[44]: from extract_feature import compute_or_load_features

# compute or load features
hog_train_features, hog_test_features = compute_or_load_features(x_train,
    ↪x_test, "hog")

# run knn
hog_knn_classifier = run_nearest_neighbor(hog_train_features, y_train,
    ↪hog_test_features, y_test)

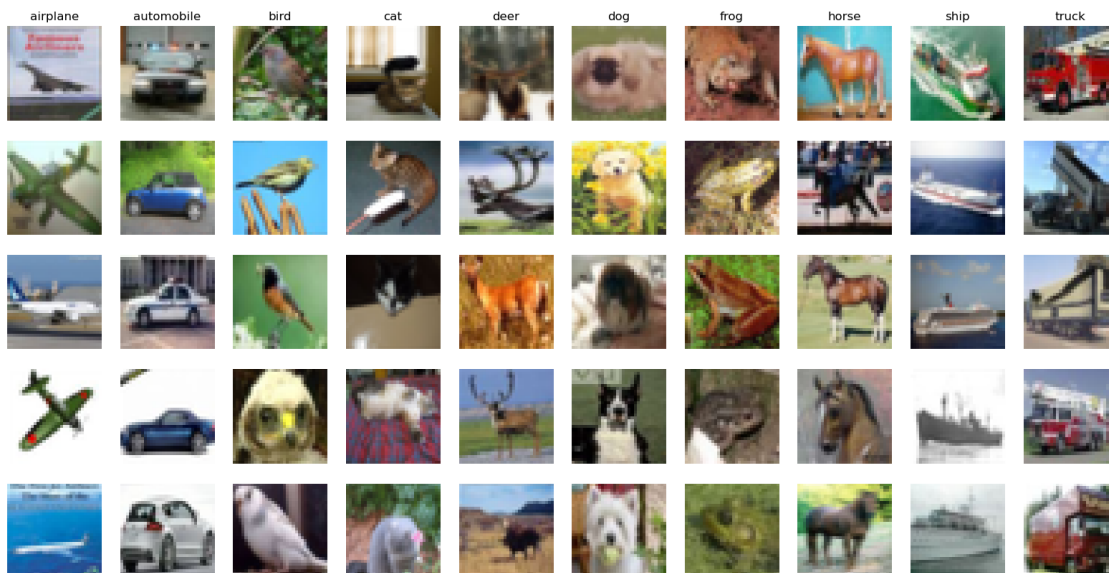
100%|      | 50000/50000 [00:30<00:00, 1656.55it/s]
100%|      | 10000/10000 [00:05<00:00, 1731.64it/s]

=====> Done with computation of HoG features
=====> Saved train and test features to /Users/twilight/Downloads/cifar-
representations/features/hog.pkl
Training feature shape: (50000, 512)
Test feature shape: (10000, 512)
Nearest neighbor accuracy on the test set: 0.365700

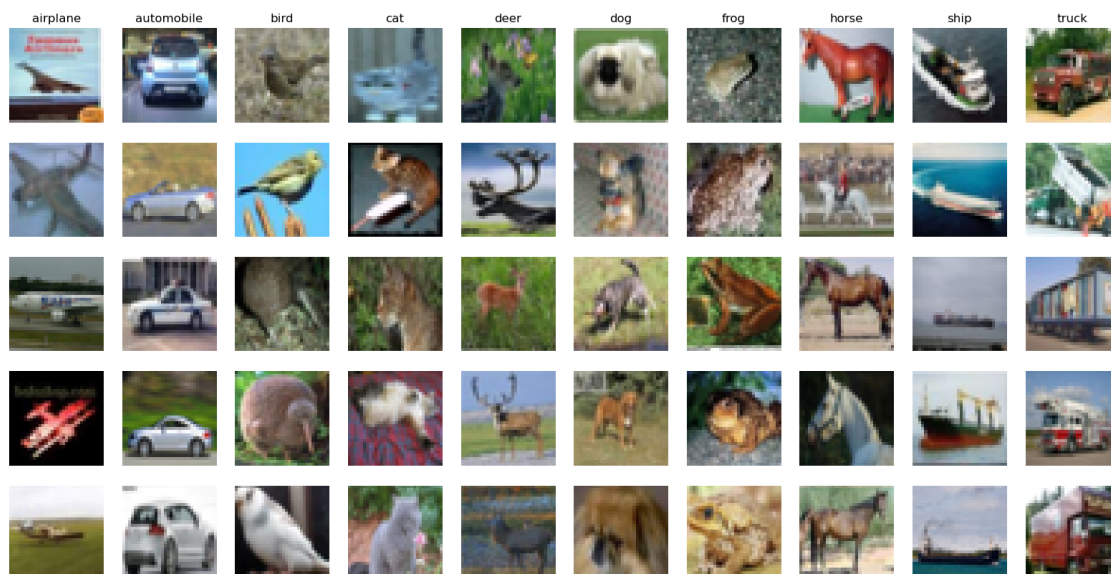
[45]: hog_predicted = hog_knn_classifier.predict(hog_test_features)
dis_hog, ind_hog = get_nearest_neighbors(hog_train_features, hog_test_features,
    ↪n_neighbors=1)

show_correct(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
    ↪hog_predicted, y_test, ind_hog, samples_per_class=5)
```

Correctly Classified Test Images

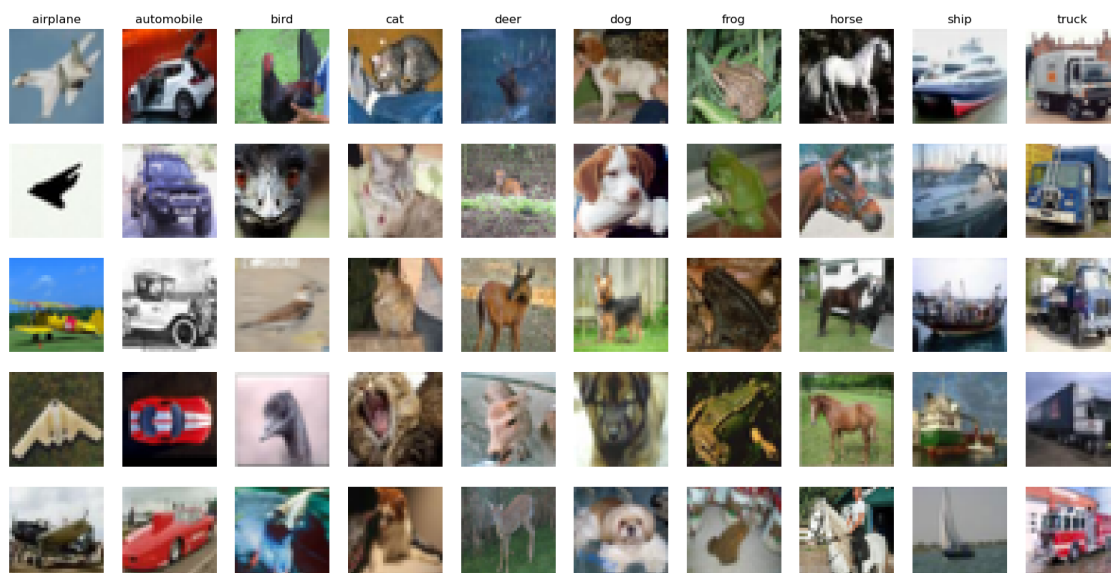


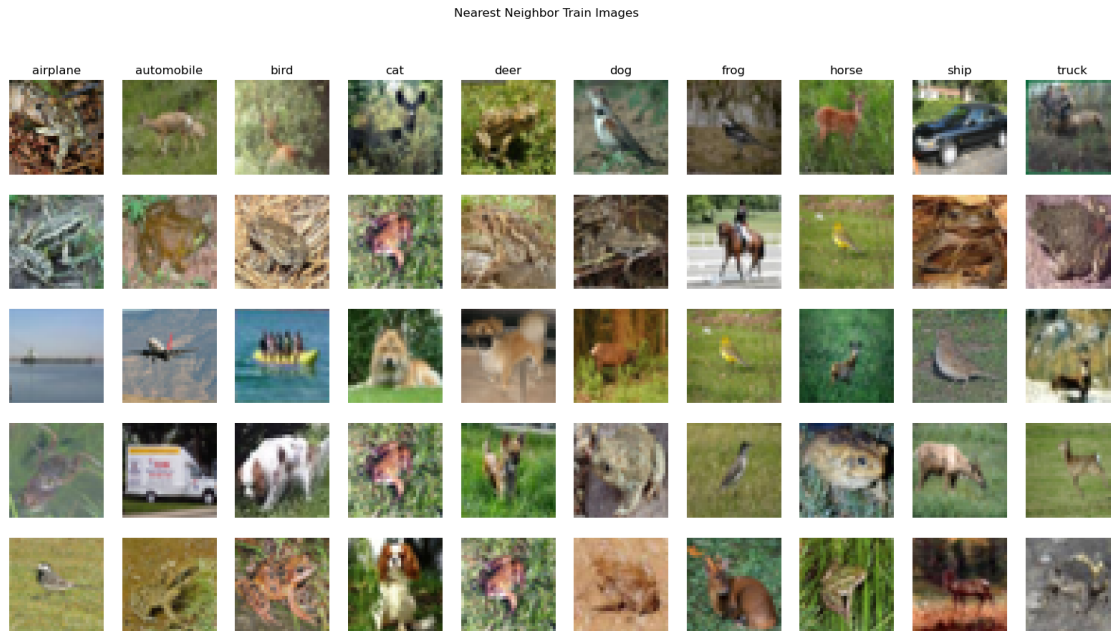
Nearest Neighbor Train Images



```
[46]: show_incorrect(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
→hog_predicted, y_test, ind_hog, samples_per_class=5)
```

Incorrectly Classified Test Images





1.2.3 Nearest neighbor on CNN features

Over the past decade, deep convolutional neural networks (CNNs) have become building blocks in a wide range of computer vision tasks. A CNN trained on a large scale image classification task learns to extract spatial hierarchies of features from edges to object parts [2].

In this problem, we will explore the representations learned by CNNs on different layers. We will be using [VGG](#) as our convolutional network architecture. A VGG11 model pretrained on CIFAR-10 can be found at `/models/vgg11_bn.pt`, so you are **NOT REQUIRED** to train the neural network. And since we only need to do forward pass through the network (no backpropagation), a CPU is enough for our purpose.

A VGG network is composed of a sequence of convolutional layers, pooling layers, and fully connected layers. To clearly understand its architecture, we provide you with a helper function visualizing the layers and input/output shapes. The following code feeds a batch of $32 \times 32 \times 3$ images into VGG11 network and do a forward pass. Run the code and check the summary.

```
[48]: from torchinfo import summary
      from vgg_network import vgg11_bn

      vgg_model = vgg11_bn(pretrained=False)
      summary(vgg_model, input_size=(16, 3, 32, 32))
```

```
[48]: =====
      =====
      Layer (type:depth-idx)                Output Shape                Param #
      =====
      =====
```

VGG	[16, 10]	--
Sequential: 1-1	[16, 512, 1, 1]	--
Conv2d: 2-1	[16, 64, 32, 32]	1,792
BatchNorm2d: 2-2	[16, 64, 32, 32]	128
ReLU: 2-3	[16, 64, 32, 32]	--
MaxPool2d: 2-4	[16, 64, 16, 16]	--
Conv2d: 2-5	[16, 128, 16, 16]	73,856
BatchNorm2d: 2-6	[16, 128, 16, 16]	256
ReLU: 2-7	[16, 128, 16, 16]	--
MaxPool2d: 2-8	[16, 128, 8, 8]	--
Conv2d: 2-9	[16, 256, 8, 8]	295,168
BatchNorm2d: 2-10	[16, 256, 8, 8]	512
ReLU: 2-11	[16, 256, 8, 8]	--
Conv2d: 2-12	[16, 256, 8, 8]	590,080
BatchNorm2d: 2-13	[16, 256, 8, 8]	512
ReLU: 2-14	[16, 256, 8, 8]	--
MaxPool2d: 2-15	[16, 256, 4, 4]	--
Conv2d: 2-16	[16, 512, 4, 4]	1,180,160
BatchNorm2d: 2-17	[16, 512, 4, 4]	1,024
ReLU: 2-18	[16, 512, 4, 4]	--
Conv2d: 2-19	[16, 512, 4, 4]	2,359,808
BatchNorm2d: 2-20	[16, 512, 4, 4]	1,024
ReLU: 2-21	[16, 512, 4, 4]	--
MaxPool2d: 2-22	[16, 512, 2, 2]	--
Conv2d: 2-23	[16, 512, 2, 2]	2,359,808
BatchNorm2d: 2-24	[16, 512, 2, 2]	1,024
ReLU: 2-25	[16, 512, 2, 2]	--
Conv2d: 2-26	[16, 512, 2, 2]	2,359,808
BatchNorm2d: 2-27	[16, 512, 2, 2]	1,024
ReLU: 2-28	[16, 512, 2, 2]	--
MaxPool2d: 2-29	[16, 512, 1, 1]	--
AdaptiveAvgPool2d: 1-2	[16, 512, 1, 1]	--
Sequential: 1-3	[16, 10]	--
Linear: 2-30	[16, 4096]	2,101,248
ReLU: 2-31	[16, 4096]	--
Dropout: 2-32	[16, 4096]	--
Linear: 2-33	[16, 4096]	16,781,312
ReLU: 2-34	[16, 4096]	--
Dropout: 2-35	[16, 4096]	--
Linear: 2-36	[16, 10]	40,970

```

=====
Total params: 28,149,514
Trainable params: 28,149,514
Non-trainable params: 0
Total mult-adds (Units.GIGABYTES): 2.75
=====

```

```

=====
Input size (MB): 0.20
Forward/backward pass size (MB): 39.85
Params size (MB): 112.60
Estimated Total Size (MB): 152.64
=====
=====

```

1.2.4 Nearest neighbor on pretrained CNN features

As shown in the summary above, a VGG net consists of three components: 1. A sequence of convolutional blocks (1-1 and 1-2) 2. Two fully connected blocks (2-30 to 2-35) 3. A single fully connected layer (2-36)

The first two components together act as a feature extractor. The convolutional blocks extract the low-level and intermediate-level features, and the fully connected blocks extract the high-level features. The third component is a single linear layer mapping the feature vectors to the classes therefore can be viewed as a linear classifier.

In this problem, we will experiment with features extracted by a pretrained VGG net at two specific layers: 1. `last_conv`: AdaptiveAvgPool2d (1-2) 2. `last_fc`: ReLU (2-34)

Train and test a nearest neighbor classifier on pretrained VGG features at these two layers respectively (Set parameter `layer` of function `compute_or_load_features` to `last_conv` or `last_fc`). Report the test accuracies with these two representations. For `last_fc`, show the first five correctly classified images (with nearest neighbors) and the first five incorrectly classified.

```

[49]: from extract_feature import compute_or_load_features

# compute or load features
pretrained_cnn_last_conv_train_features, pretrained_cnn_last_conv_test_features =
    compute_or_load_features(x_train, x_test, "pretrained_cnn", "last_conv")

# run knn
pretrained_cnn_last_conv_knn_classifier =
    run_nearest_neighbor(pretrained_cnn_last_conv_train_features, y_train,
    pretrained_cnn_last_conv_test_features, y_test)

```

```

100%|      | 500/500 [01:34<00:00,  5.30it/s]
100%|      | 100/100 [00:20<00:00,  4.99it/s]

=====> Done with computation of CNN features
=====> Saved train and test features to /Users/twilight/Downloads/cifar-
representations/features/pretrained_cnn_last_conv.pkl
Training feature shape: (50000, 512)
Test feature shape: (10000, 512)
Nearest neighbor accuracy on the test set: 0.920000

```

```

[50]: pretrained_last_conv_predicted = pretrained_cnn_last_conv_knn_classifier.
    predict(pretrained_cnn_last_conv_test_features)

```

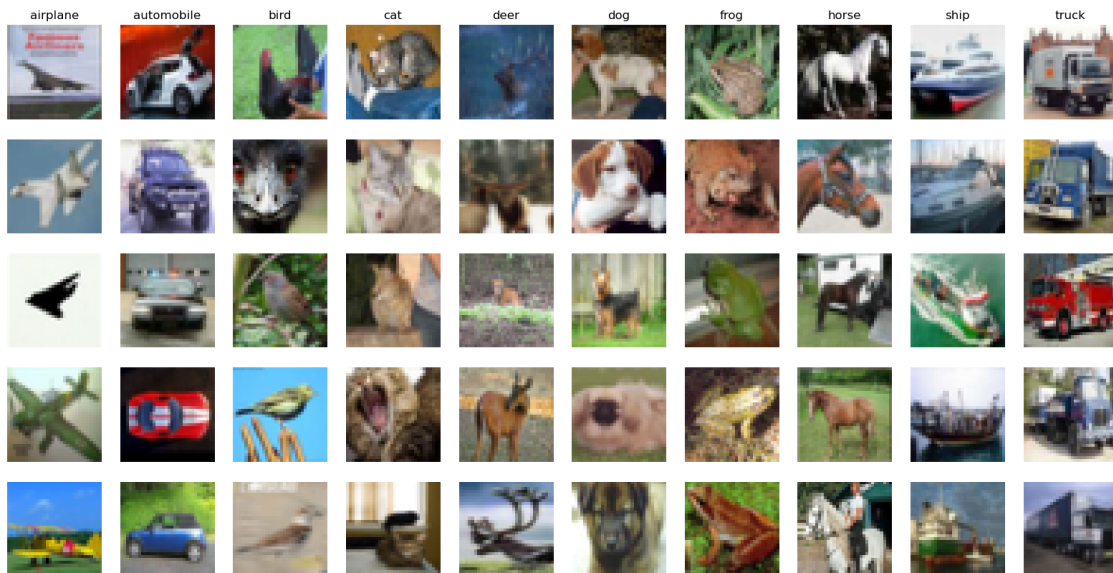
```

dis_pret_last_conv, ind_pret_last_conv = ␣
    ↪ get_nearest_neighbors(pretrained_cnn_last_conv_train_features, ␣
    ↪ pretrained_cnn_last_conv_test_features, n_neighbors=1)

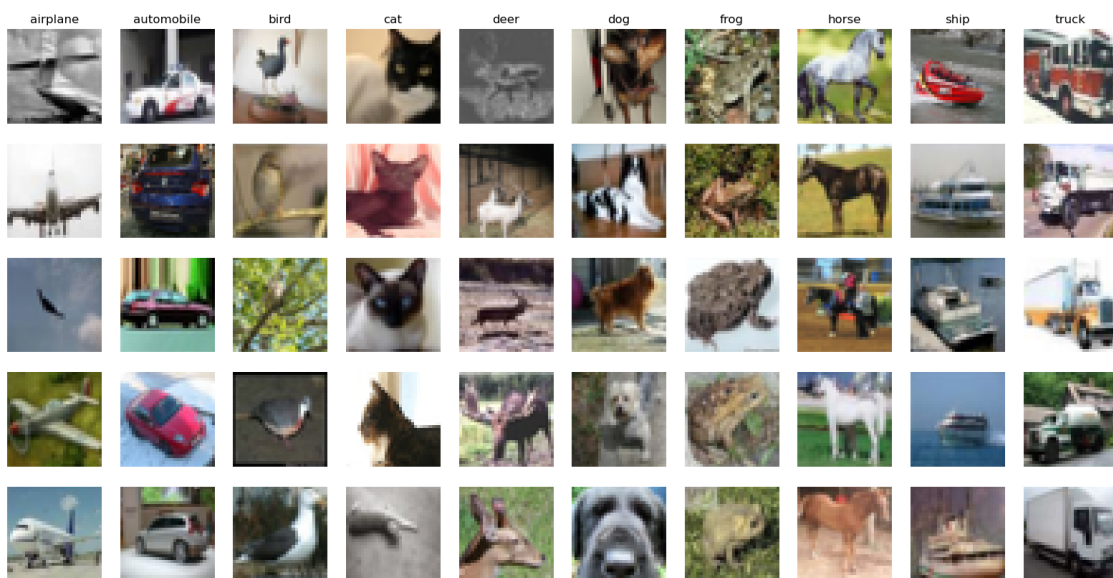
show_correct(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1), ␣
    ↪ pretrained_last_conv_predicted, y_test, ind_pret_last_conv, ␣
    ↪ samples_per_class=5)

```

Correctly Classified Test Images

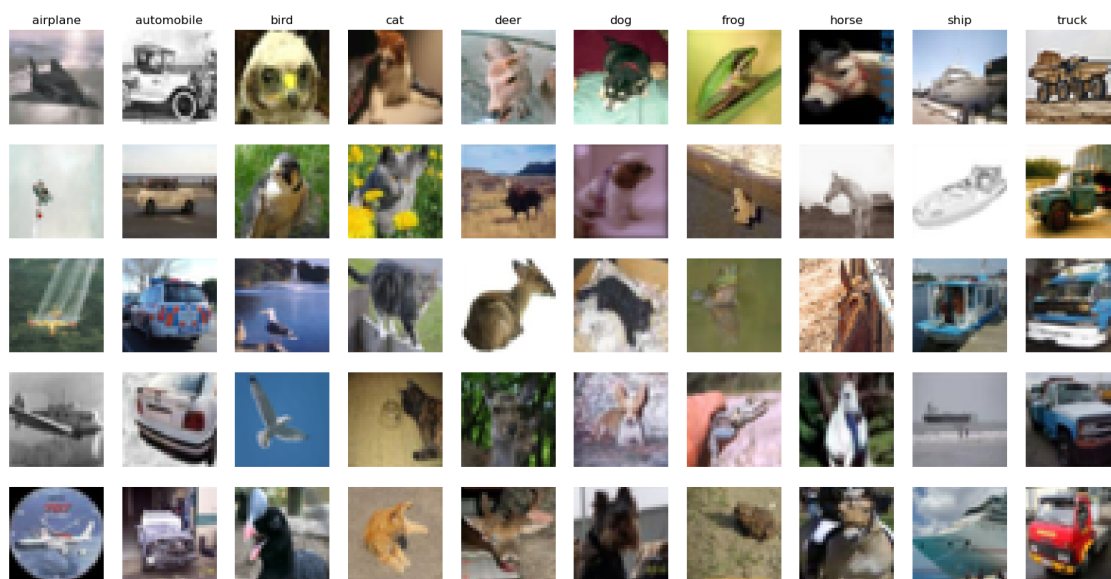


Nearest Neighbor Train Images

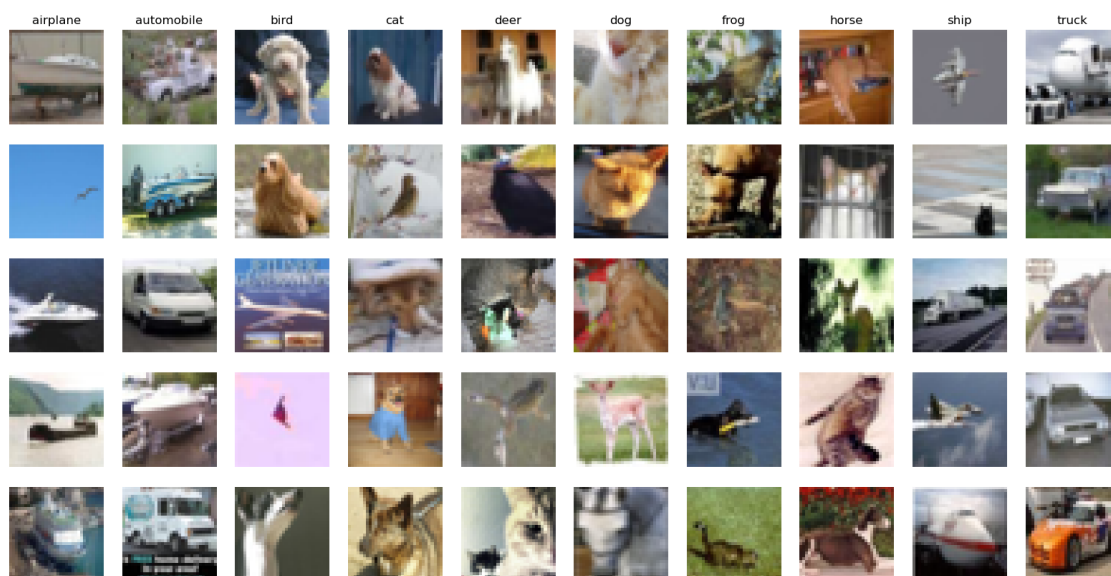



```
[51]: show_incorrect(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
↳pretrained_last_conv_predicted, y_test, ind_pret_last_conv,
↳samples_per_class=5)
```

Incorrectly Classified Test Images



Nearest Neighbor Train Images



```
[52]: from extract_feature import compute_or_load_features

# compute or load features
pretrained_cnn_last_fc_train_features, pretrained_cnn_last_fc_test_features = \
    ↪compute_or_load_features(x_train, x_test, "pretrained_cnn", "last_fc")

# run knn
pretrained_cnn_last_fc_knn_classifier = \
    ↪run_nearest_neighbor(pretrained_cnn_last_fc_train_features, y_train, \
    ↪pretrained_cnn_last_fc_test_features, y_test)
```

100%| | 500/500 [01:41<00:00, 4.95it/s]

100%| | 100/100 [00:23<00:00, 4.22it/s]

=====> Done with computation of CNN features

=====> Saved train and test features to /Users/twilight/Downloads/cifar-
representations/features/pretrained_cnn_last_fc.pkl

Training feature shape: (50000, 4096)

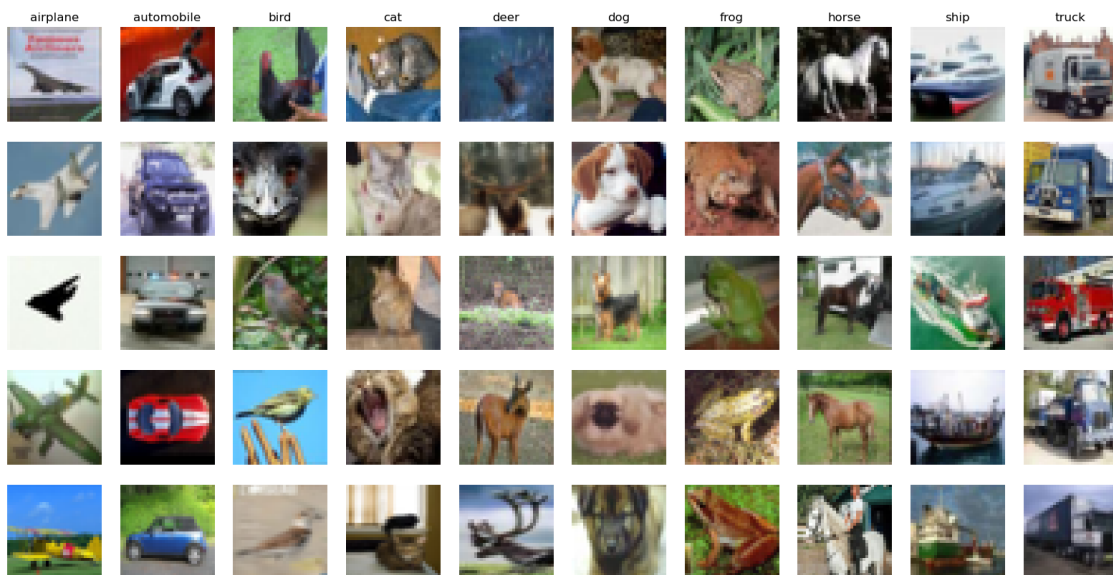
Test feature shape: (10000, 4096)

Nearest neighbor accuracy on the test set: 0.920700

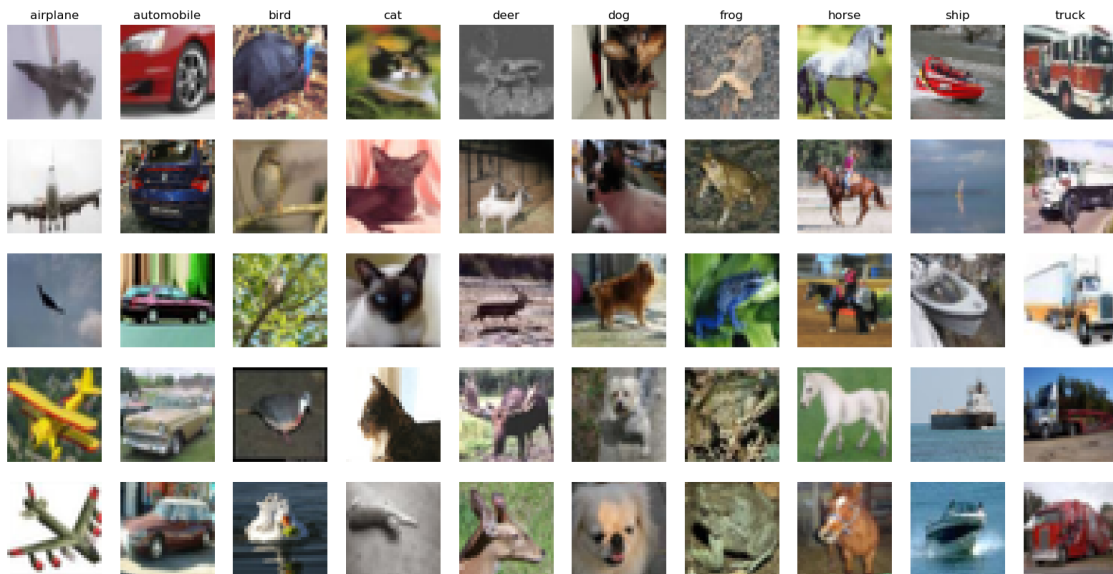
```
[53]: pretrained_last_fc_predicted = pretrained_cnn_last_fc_knn_classifier.
    ↪predict(pretrained_cnn_last_fc_test_features)
dis_pret_last_fc, ind_pret_last_fc = \
    ↪get_nearest_neighbors(pretrained_cnn_last_fc_train_features, \
    ↪pretrained_cnn_last_fc_test_features, n_neighbors=1)

show_correct(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1), \
    ↪pretrained_last_fc_predicted, y_test, ind_pret_last_fc, samples_per_class=5)
```

Correctly Classified Test Images

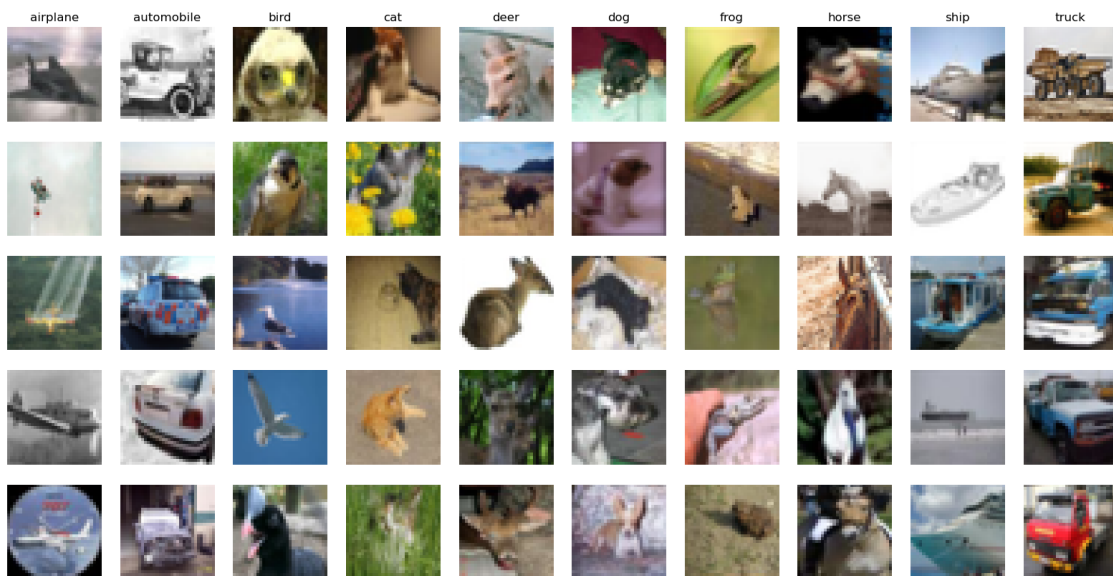


Nearest Neighbor Train Images

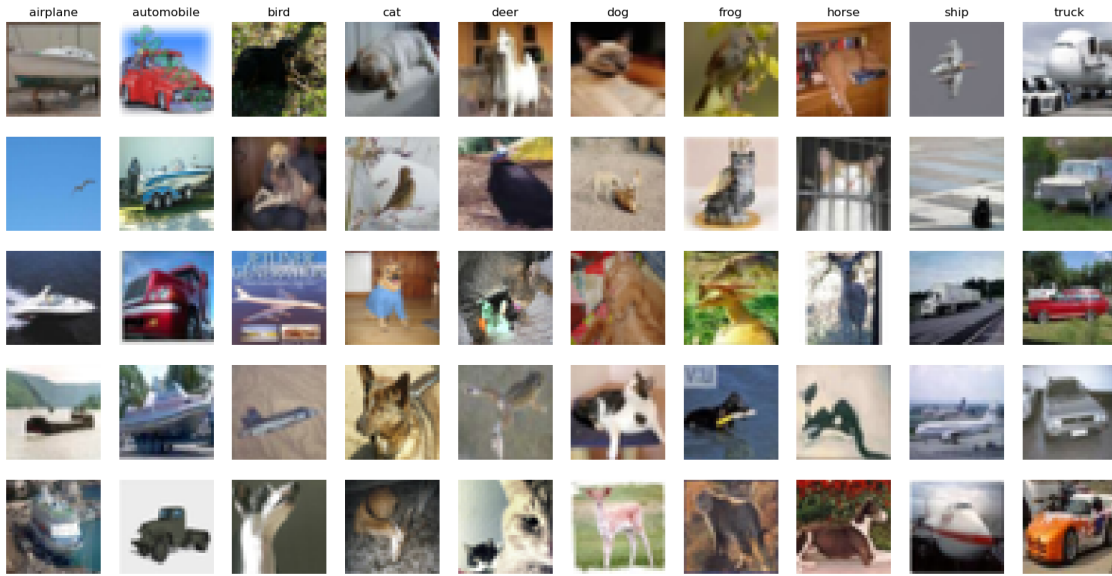


```
[54]: show_incorrect(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
↳ pretrained_last_fc_predicted, y_test, ind_pret_last_fc, samples_per_class=5)
```

Incorrectly Classified Test Images



Nearest Neighbor Train Images



1.2.5 Alignment between nearest neighbor classifier and pretrained CNN model

In the following block, call function `test_pretrained_vgg` from `vgg_network.py`. The function will test the pretrained VGG net on the test images of CIFAR-10 and return the test accuracy. Report this accuracy and compare it with the test accuracy of nearest neighbor classifier in pre-trained `last conv` and `last fc` feature space. In which space does the nearest neighbor classifier behave the most similarly to a pretrained CNN model?

```
[55]: from vgg_network import test_pretrained_vgg

test_acc = test_pretrained_vgg(x_test, y_test)
print('Accuracy on the test images: ', test_acc)
```

Accuracy on the test images: 0.9225

1.2.6 Nearest neighbor on random CNN features

The excellent feature extraction ability of ConvNets enables them to solve computer vision problems in a universal way. One may think that this is because ConvNets are generally trained on large datasets of images. But is this the only reason? To answer this question, a recent work Deep Image Prior [1] shows that the structure of a CNN is sufficient to capture a great deal of low-level image statistics prior to any learning. A randomly initialized fully-convolutional neural network is already able to achieve good results in standard image reconstruction problems such as denoising, super-resolution, and inpainting.

In this problem, we will investigate whether the inductive bias captured by the ConvNets also helps for image classification problems.

You will train and test the nearest neighbor classifier on the features extracted by a randomly initialized VGG network. To do: Experiment with two feature spaces `last_conv` and `last_fc`, and report the test accuracies.

```
[ ]: from extract_feature import compute_or_load_features

# compute or load features
random_cnn_last_conv_train_features, random_cnn_last_conv_test_features = \
    ↪compute_or_load_features(x_train, x_test, "random_cnn", "last_conv")

# run knn
random_cnn_last_conv_knn_classifier = \
    ↪run_nearest_neighbor(random_cnn_last_conv_train_features, y_train, \
    ↪random_cnn_last_conv_test_features, y_test) # acc will vary due to random \
    ↪initialization
```

```
100%|      | 500/500 [06:18<00:00, 1.32it/s]
```

```
100%|      | 100/100 [00:58<00:00, 1.71it/s]
```

```
=====> Done with computation of CNN features
```

```
=====> Saved train and test features to /Users/twilight/Downloads/cifar-
representations/features/random_cnn_last_conv.pkl
```

```
Training feature shape: (50000, 512)
```

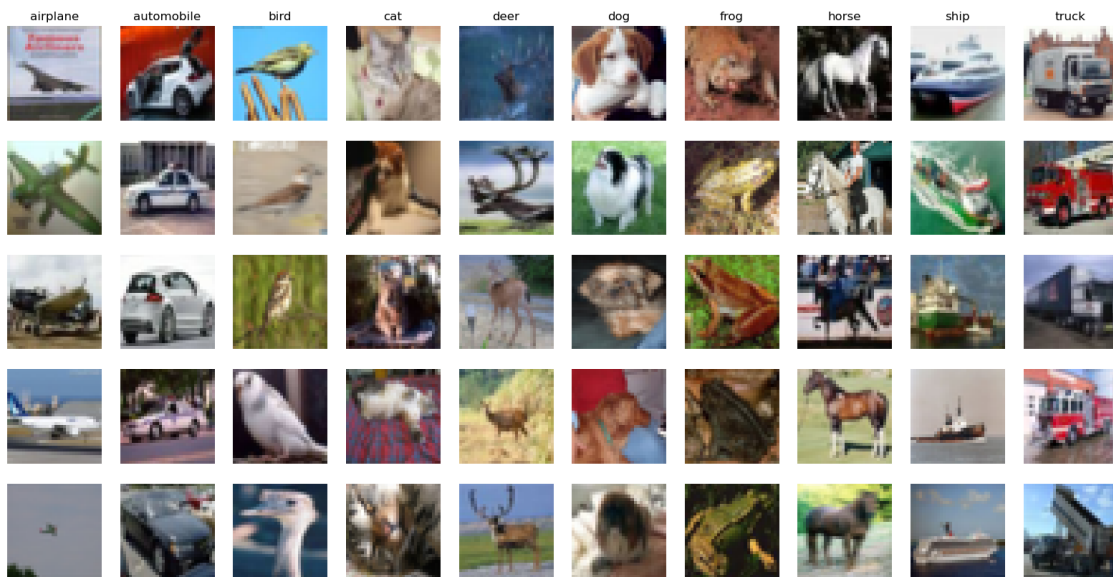
```
Test feature shape: (10000, 512)
```

```
Nearest neighbor accuracy on the test set: 0.396900
```

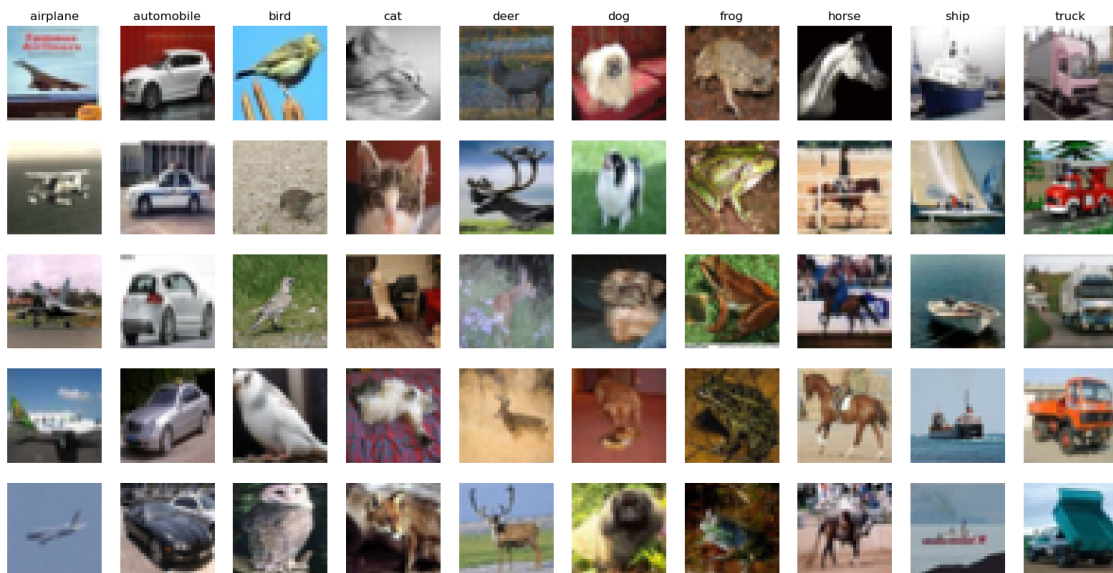
```
[58]: rand_last_conv_predicted = random_cnn_last_conv_knn_classifier.
    ↪predict(random_cnn_last_conv_test_features)
dis_rand_last_conv, ind_rand_last_conv = \
    ↪get_nearest_neighbors(random_cnn_last_conv_train_features, \
    ↪random_cnn_last_conv_test_features, n_neighbors=1)

show_correct(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1), \
    ↪rand_last_conv_predicted, y_test, ind_rand_last_conv, samples_per_class=5)
```

Correctly Classified Test Images

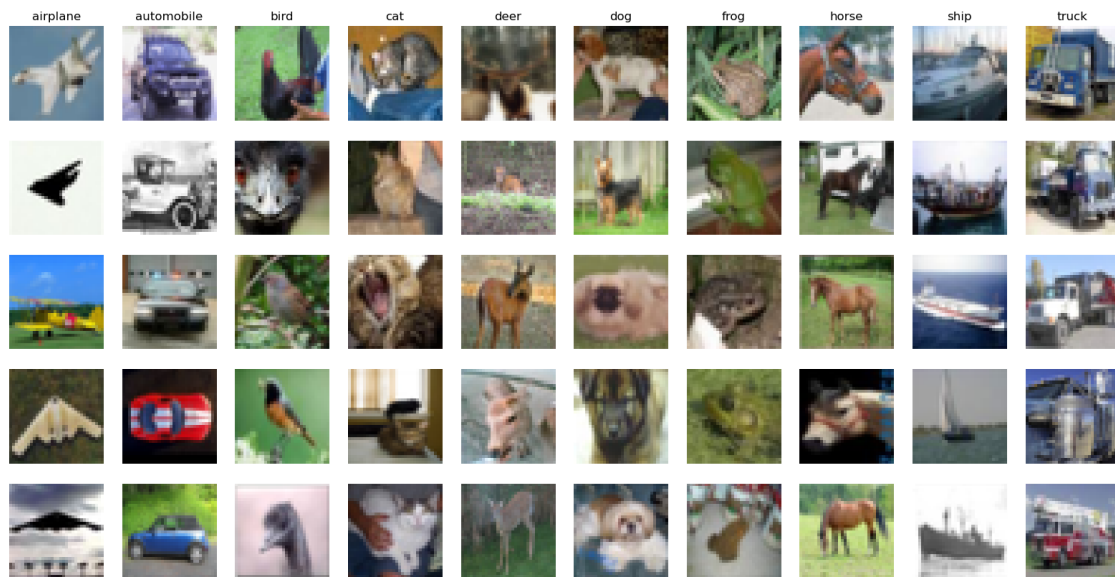


Nearest Neighbor Train Images

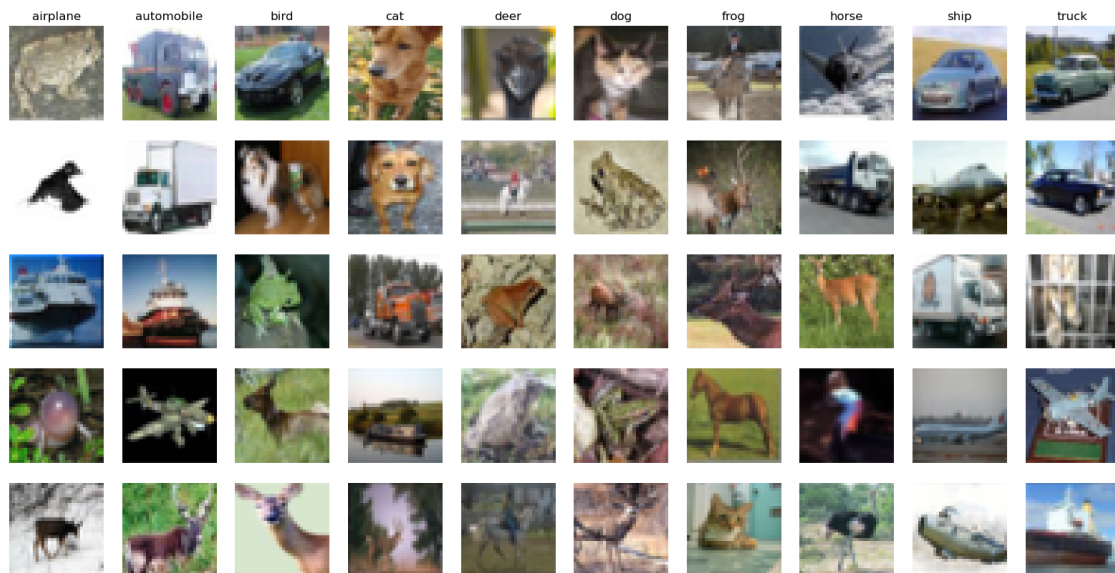


```
[59]: show_incorrect(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
    ↪ rand_last_conv_predicted, y_test, ind_rand_last_conv, samples_per_class=5)
```

Incorrectly Classified Test Images



Nearest Neighbor Train Images



```
[ ]: from extract_feature import compute_or_load_features

# compute or load features
random_cnn_last_fc_train_features, random_cnn_last_fc_test_features = \
    compute_or_load_features(x_train, x_test, "random_cnn", "last_fc")
```



```
# run knn
random_cnn_last_fc_knn_classifier =
↳run_nearest_neighbor(random_cnn_last_fc_train_features, y_train,
↳random_cnn_last_fc_test_features, y_test) # acc will vary due to random
↳initialization
```

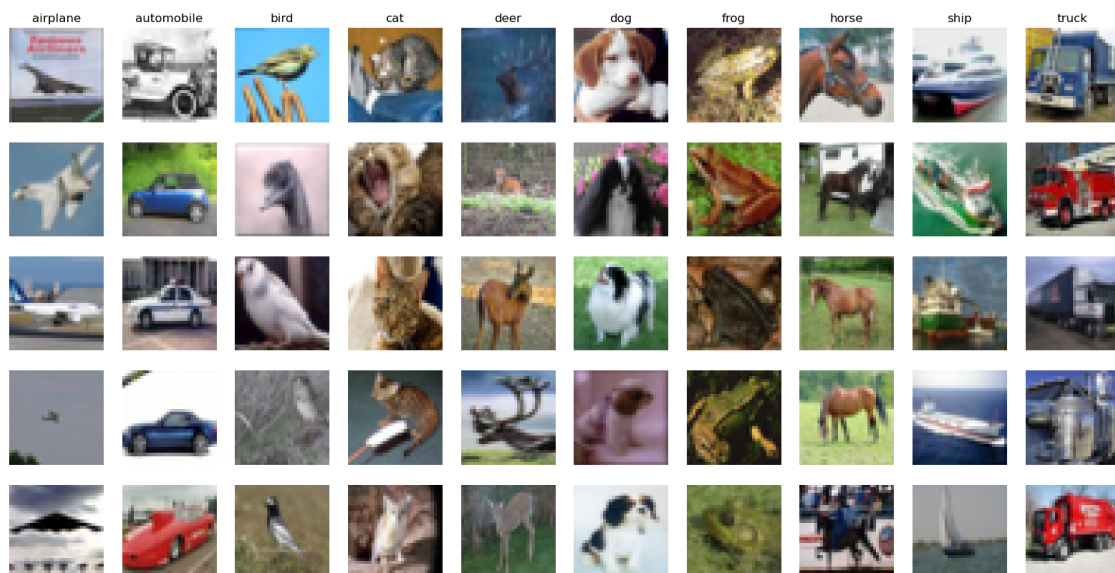
```
100%|      | 500/500 [01:41<00:00, 4.95it/s]
100%|      | 100/100 [00:20<00:00, 4.97it/s]
```

```
=====> Done with computation of CNN features
=====> Saved train and test features to /Users/twilight/Downloads/cifar-
representations/features/random_cnn_last_fc.pkl
Training feature shape: (50000, 4096)
Test feature shape: (10000, 4096)
Nearest neighbor accuracy on the test set: 0.374100
```

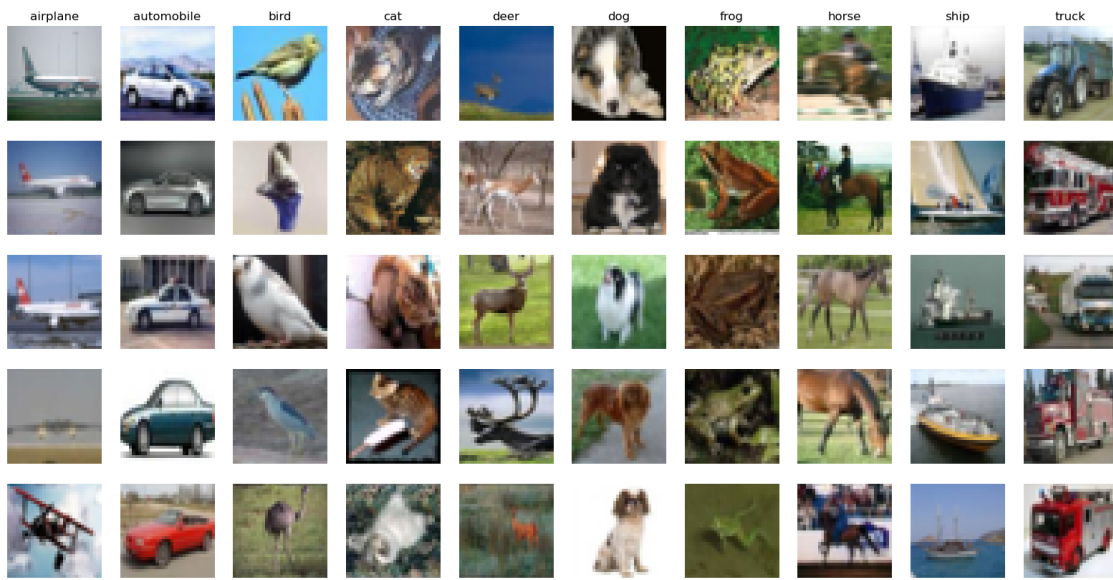
```
[61]: rand_last_fc_predicted = random_cnn_last_fc_knn_classifier.
↳predict(random_cnn_last_fc_test_features)
dis_rand_last_fc, ind_rand_last_fc =
↳get_nearest_neighbors(random_cnn_last_fc_train_features,
↳random_cnn_last_fc_test_features, n_neighbors=1)

show_correct(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
↳rand_last_fc_predicted, y_test, ind_rand_last_fc, samples_per_class=5)
```

Correctly Classified Test Images

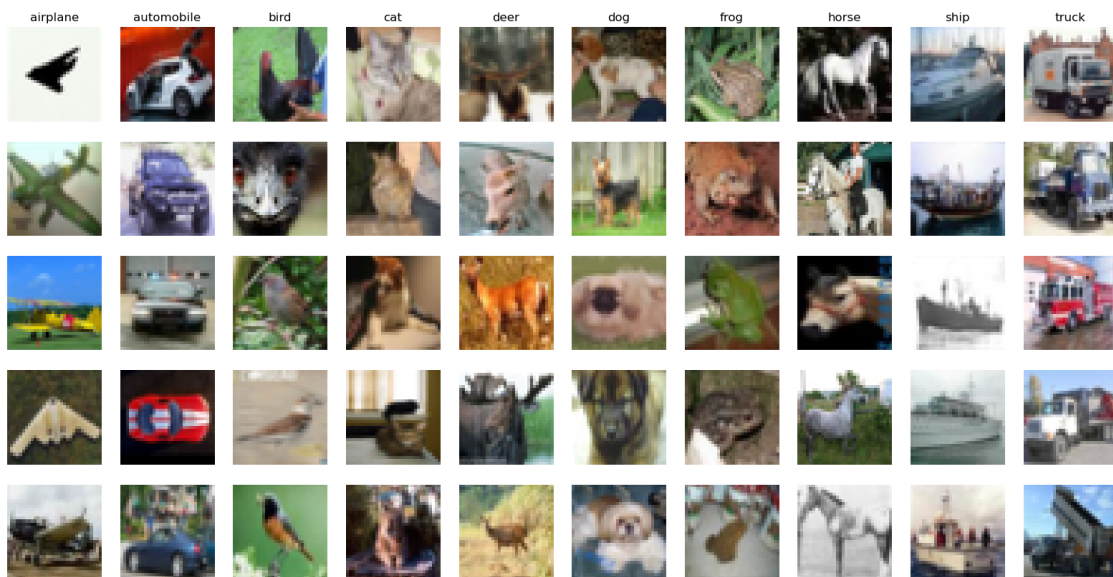


Nearest Neighbor Train Images

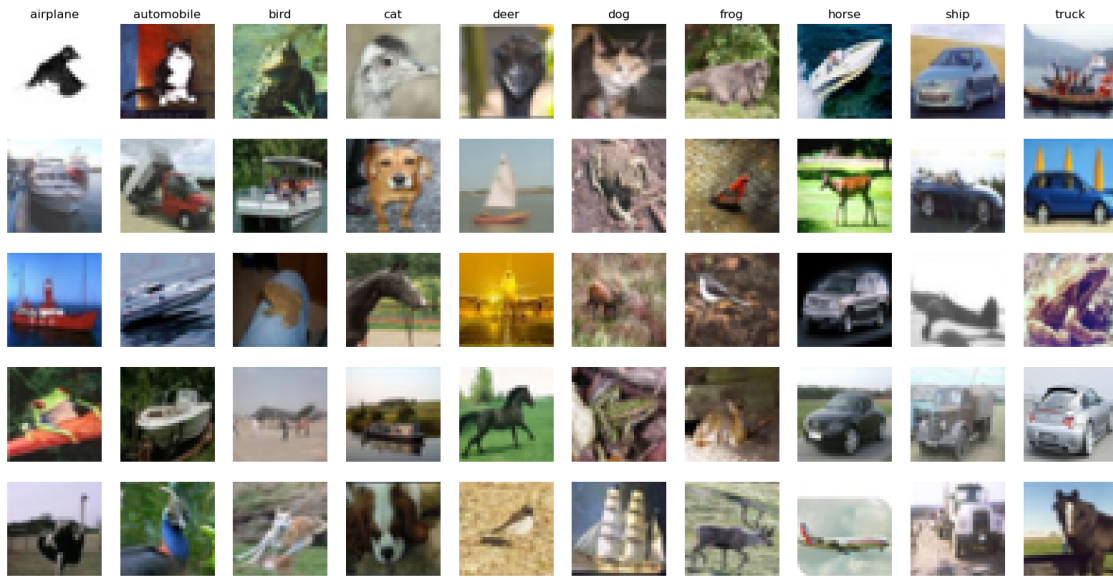


```
[62]: show_incorrect(x_train.transpose(0, 2, 3, 1), x_test.transpose(0, 2, 3, 1),
↳ rand_last_fc_predicted, y_test, ind_rand_last_fc, samples_per_class=5)
```

Incorrectly Classified Test Images



Nearest Neighbor Train Images



Compare the test accuracies of nearest neighbor classifier on random `last conv` features and HOG features. In which representation space does it perform better?

Compare the test accuracies of nearest neighbor classifier on random `last conv` features and random `last fc` features. In which representation space does it perform better?

1.3 References

1. D. Ulyanov, A. Vedaldi, and V. Lempitsky, [Deep Image Prior](#), CVPR 2018.
2. M. Zeiler and R. Fergus, [Visualizing and Understanding Deep Neural Networks](#), ECCV 2014.